VISIÓN ARTIFICIAL · LIC. EN INTELIGENCIA ARTIFICIAL · 7° SEMESTRE

Práctica 3 · Segmentación

Separar los objetos del fondo en video: umbrales, bordes, agrupación, conocimiento del dominio, movimiento, Hough y contornos activos

Modalidad	Individual, en clase y junto con el docente (programación en vivo)
Duración	2 horas en clase: pasos 1–5. **Pasos 6 y 7 de tarea** si no se alcanzan
Temas	2.4 Conocimiento del dominio · 2.5 Fronteras · 2.6 Umbralización · 2.7 Agrupación · 2.8 Movimiento · 2.9 Hough y contornos activos
Herramientas	Python 3.10+ · OpenCV · NumPy · Matplotlib · scikit-image · webcam
Entregable	Los 7 scripts + reporte con capturas, tabla de IoU y respuestas (sección 10)

Objetivo

Que programes las principales familias de **segmentación** sobre video, que veas en qué falla cada una y que las compares con una **métrica objetiva** (IoU) contra la respuesta correcta.

Cómo está organizada la práctica

Todos los scripts usan la **misma `[BASE]` de video de la Práctica 2**, sin cambios. Lo nuevo es que cada técnica se escribe como **funciones** (`segmentar_otsu`, `segmentar_bordes`, …) y los pasos siguientes las **importan** en lugar de copiarlas: por eso todos los scripts deben estar en la misma carpeta.

El orden no es el del temario, sino de lo más simple a lo más elaborado:

Paso	Tema	Script	Qué vas a ver
1	2.6 Umbralización	`p3_paso1_umbral.py`	Global, Otsu y adaptativo; limpieza morfológica
2	2.5 Fronteras	`p3_paso2_bordes.py`	Sobel, Canny y cómo pasar de bordes a regiones
3	2.7 Agrupación	`p3_paso3_agrupacion.py`	k-means de color y crecimiento de regiones con un clic
4	2.4 Conocimiento del dominio	`p3_paso4_dominio.py`	Color + tamaño + forma: solo se cuenta el objeto buscado
5	2.8 Movimiento	`p3_paso5_movimiento.py`	Diferencia de frames, fondo promedio y MOG2
6	2.9 Hough y contornos activos	`p3_paso6_hough_snake.py`	Líneas y círculos de Hough; una snake que se pega al borde
7	Comparativa	`p3_paso7_comparativa.py`	IoU de todas las técnicas contra la máscara verdadera

En cada paso: **Lo que se verá**, **Qué tienes que hacer** y **Resultado esperado**.

Contenido

- Objetivo
- Cómo está organizada la práctica
- 1. Preparar el entorno
- 2. Conceptos clave
- 3. Paso 1 · Umbralización (2.6)
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 1
- 4. Paso 2 · Segmentación por fronteras (2.5)
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 2
- 5. Paso 3 · Agrupación de píxeles (2.7)
  - Lo que se verá
  - Qué tienes que hacer
  - Resultado esperado
  - Preguntas del paso 3
- 6. Paso 4 · Conocimiento del dominio (2.4)

    - Lo que se verá
    - Qué tienes que hacer
    - Resultado esperado
    - Preguntas del paso 4
- 7. Paso 5 · Movimiento (2.8)
    - Lo que se verá
    - Qué tienes que hacer
    - Resultado esperado
    - Preguntas del paso 5
- 8. Paso 6 · Hough y contornos activos (2.9) · *tarea si no se alcanza*
    - Lo que se verá
    - Qué tienes que hacer
    - Resultado esperado
    - Preguntas del paso 6
- 9. Paso 7 · Comparativa con IoU · *tarea si no se alcanza*
    - Lo que se verá
    - Qué tienes que hacer
    - Resultado esperado
    - Preguntas del paso 7
- 10. Retos y entregable
    - Retos
    - Entregable
- 11. Errores frecuentes
    - Referencia rápida

1. Preparar el entorno

Mismo entorno de la Práctica 2, con `scikit-image` instalado (`pip install -r requirements.txt`). Comprueba:

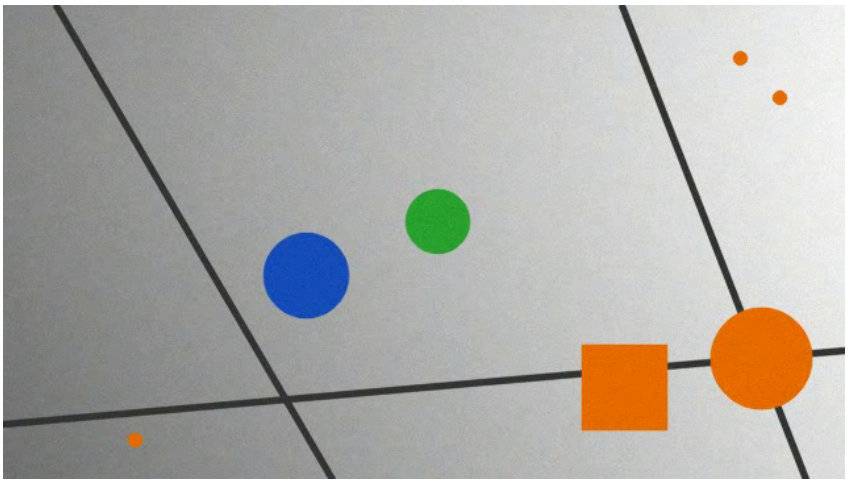
```
python -c "import cv2, skimage; print(cv2.__version__, skimage.__version__)"
```

Estructura de la carpeta:

```
practica03/
├── recursos/
│   ├── video_sintetico.mp4     <- el mismo video de la P2
│   └── mascaras.npz            <- la segmentación CORRECTA de cada frame (para el paso 7)
├── requirements.txt
├── p3_paso1_umbral.py
├── ...
└── p3_paso7_comparativa.py
```

Trabaja primero con `--video` y después con tu webcam. El video sintético está diseñado para que cada técnica muestre su fortaleza y su debilidad:

- **Iluminación dispareja:** el fondo va de gris oscuro (izquierda) a casi blanco (derecha).
- **Tres líneas rectas** oscuras en el fondo.
- **Una pelota naranja** (el objeto buscado), **un cuadrado naranja** (mismo color, otra forma) y **tres puntos naranjas** pequeños.
- **Un círculo azul que se detiene** a la mitad del video y un **círculo verde** que sube y baja.



Frame 160 del video sintético

Con la webcam, usa un objeto de **color sólido y llamativo** (pelota, tapa, post-it) sobre un fondo liso.

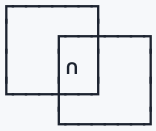
2. Conceptos clave

- **Segmentar** es asignar cada píxel a "objeto" o "fondo" (o a una de varias regiones). El resultado es una **máscara**: una imagen binaria con 255 en el objeto y 0 en el fondo.
- Cada familia usa una propiedad distinta del objeto:

Familia	Qué propiedad usa	Falla cuando…
Umbralización	**Intensidad** de cada píxel	La iluminación no es pareja o el objeto y el fondo tienen el mismo gris
Fronteras	**Cambios bruscos** de intensidad	Los bordes tienen huecos o hay bordes que no son del objeto
Agrupación	**Parecido** entre píxeles (color, posición)	Hay objetos distintos del mismo color
Dominio	Lo que **sabemos** del objeto (color, tamaño, forma)	El objeto no cumple las reglas (se traslapa, cambia de color)
Movimiento	**Cambio en el tiempo**	El objeto se detiene o la cámara se mueve
Modelos (Hough, snakes)	Una **forma** esperada	La forma no coincide con el modelo

- **Morfología:** operaciones sobre máscaras con un "pincel" (*elemento estructurante*). La **apertura** borra lo más pequeño o delgado que el pincel; el **cierre** tapa huecos y une partes cercanas.
- **IoU (*Intersection over Union*):** área en común entre la máscara obtenida y la correcta, dividida entre el área de su unión. 1 = perfecta; 0 = nada en común.

máscara obtenida A máscara correcta B



$$IoU = |A \cap B| \,/\, |A \cup B|$$
$$Dice = 2 \cdot |A \cap B| \,/\, (|A| + |B|)$$

3. Paso 1 · Umbralización (2.6)

Lo que se verá

Un **umbral** T decide: si el gris es menor que T, el píxel es objeto; si no, es fondo (usamos `THRESH_BINARY_INV` porque nuestros objetos son más oscuros que el fondo).

- **Global:** un solo T, elegido a mano, para toda la imagen.
- **Otsu:** un solo T, pero calculado automáticamente: el que mejor separa el histograma en dos grupos.
- **Adaptativo:** un T distinto para cada píxel: el promedio de su vecindad menos una constante C. Funciona con iluminación dispareja.

Qué tienes que hacer

1. Crea `p3_paso1_umbral.py` con la `[BASE]` de video de la Práctica 2 (cópiala sin cambios).
2. Agrega las funciones de segmentación:

```python
def preparar_gris(frame, sigma=1.5):
    """Gris suavizado: el ruido hace que píxeles vecinos caigan de lados distintos del umbral."""
    gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    return cv2.GaussianBlur(gris, (0, 0), sigma) if sigma > 0 else gris


def segmentar_global(gris, t):
    """Objeto = píxeles MÁS OSCUROS que t (INV invierte: oscuro -> 255). Un solo T para toda la imagen."""
    _, mascara = cv2.threshold(gris, t, 255, cv2.THRESH_BINARY_INV)
    return mascara


def segmentar_otsu(gris):
    """Otsu elige T solo: el que mejor separa el histograma en dos grupos. Devuelve (máscara, T)."""
    t, mascara = cv2.threshold(gris, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)   # el 0 se ignora
    return mascara, t


def segmentar_adaptativo(gris, bloque=101, c=10):
    """Un T distinto por píxel: el promedio (ponderado) de su vecindad de bloque x bloque menos c."""
    return cv2.adaptiveThreshold(gris, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, bloque, c)


def limpiar(mascara, k=5):
    """Apertura (erosión + dilatación) quita lo más delgado que k; cierre (dilatación + erosión) tapa huecos."""
    if k < 2:
        return mascara
    elemento = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))     # "pincel" circular de k x k
    abierta = cv2.morphologyEx(mascara, cv2.MORPH_OPEN, elemento)
    return cv2.morphologyEx(abierta, cv2.MORPH_CLOSE, elemento)

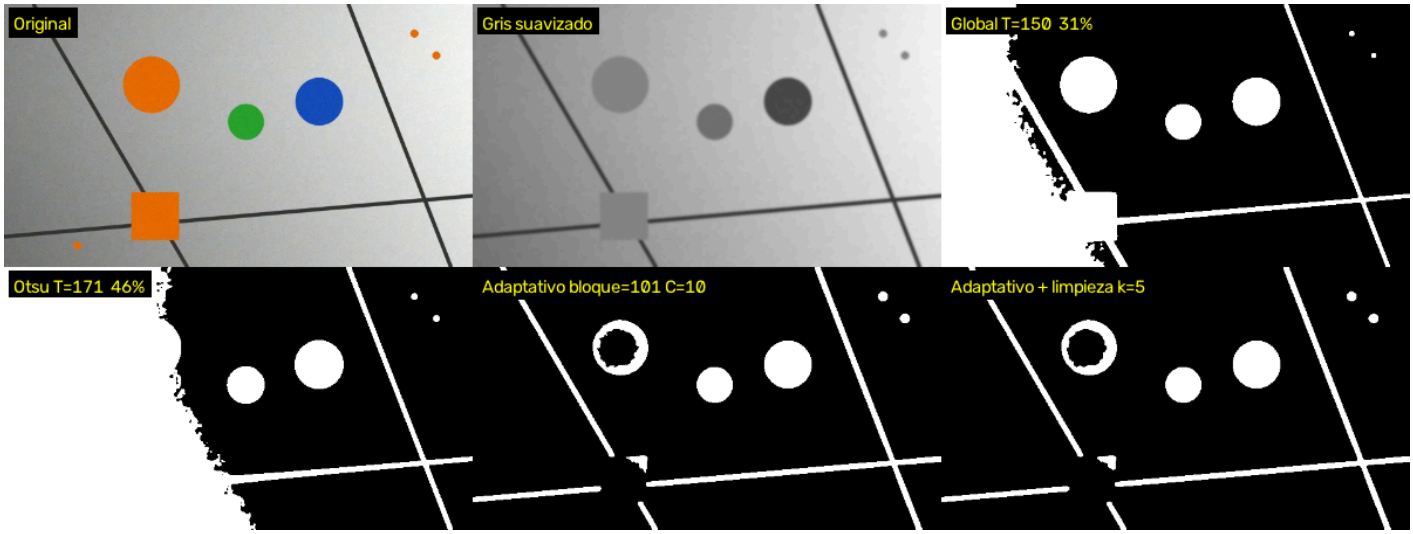

def porcentaje(mascara):
    """Qué porcentaje de la imagen quedó como objeto (255)."""
    return 100 * cv2.countNonZero(mascara) / mascara.size
```

3. Programa principal: ventana con 5 barras (`Suavizado x10` = 15, `T global` = 150, `Bloque` = 50 → 101, `C` = 10, `Limpieza` = 5) y, en el ciclo, un mosaico 2×3: original, gris suavizado, global, Otsu (con el T que eligió), adaptativo y adaptativo limpio. Pon en cada título el % de la imagen que quedó como objeto.

```
gris = preparar_gris(frame, valor("Suavizado x10", VENTANA) / 10)
bloque = 2 * max(1, valor("Bloque", VENTANA)) + 1        # impar y > 1
otsu, t_otsu = segmentar_otsu(gris)
adapt = segmentar_adaptativo(gris, bloque, c)
limpia = limpiar(adapt, k)
```

Resultado esperado

```
== Último frame ==
T global = 150   T de Otsu = 171
global      objeto =  30.6 % de la imagen
otsu        objeto =  45.8 % de la imagen
adaptativo  objeto =   8.0 % de la imagen
limpia      objeto =   7.9 % de la imagen
```



El global y Otsu se "comen" la zona oscura de la izquierda; el adaptativo encuentra los objetos en toda la imagen, pero la pelota naranja (casi del mismo gris que el fondo) queda hueca

Preguntas del paso 1

1. ¿Por qué ningún T global separa bien los objetos del fondo en el video sintético? Muévelo de 100 a 200 y describe qué pasa.
2. ¿Qué T eligió Otsu? ¿Por qué Otsu falla aquí aunque "elige el mejor T"?
3. En el adaptativo, baja el `Bloque` a 5 y súbelo a 150. ¿Qué pasa con los objetos grandes cuando el bloque es más pequeño que ellos?
4. ¿Por qué la pelota naranja sale hueca o incompleta en el adaptativo? (Pista: compara su gris con el del fondo a su alrededor.)
5. Sube `Limpieza` a 9. ¿Qué desaparece primero, las líneas o los puntos pequeños? ¿Por qué?

4. Paso 2 · Segmentación por fronteras (2.5)

Lo que se verá

Un **borde** es un cambio brusco de intensidad. Lo medimos con la **derivada**:

- **Sobel** calcula la derivada en x (Gx) y en y (Gy); la **magnitud** $\sqrt{(Gx^2 + Gy^2)}$ es la fuerza del borde.
- **Canny** deja bordes de **1 píxel** y continuos: suaviza, calcula el gradiente, adelgaza (supresión de no máximos) y decide con **dos umbrales** (histéresis): lo que está encima de "alto" es borde; lo que está entre "bajo" y "alto" solo si se conecta con un borde fuerte.
- Un borde **no es una región**: hay que **cerrar** los huecos y **rellenar** los contornos cerrados para obtener una máscara.

Qué tienes que hacer

1. Crea `p3_paso2_bordes.py` con la misma `[BASE]`.
2. **Importa** `preparar_gris` del paso 1 y agrega las funciones:

```python
from p3_paso1_umbral import preparar_gris                    # reutilizamos el paso 1 (mismo folder)


def gradiente_sobel(gris):
    """Derivadas con Sobel. CV_32F: las derivadas pueden ser negativas y uint8 no las guarda."""
    gx = cv2.Sobel(gris, cv2.CV_32F, 1, 0, ksize=3)          # cambio de izquierda a derecha (bordes verticales)
    gy = cv2.Sobel(gris, cv2.CV_32F, 0, 1, ksize=3)          # cambio de arriba a abajo (bordes horizontales)
    magnitud = cv2.magnitude(gx, gy)                         # sqrt(gx² + gy²): fuerza del borde
    return gx, gy, magnitud


def a_8bits(m):
    """Lleva cualquier matriz (float, con negativos) a 0..255 para poder verla."""
    return cv2.normalize(np.abs(m), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

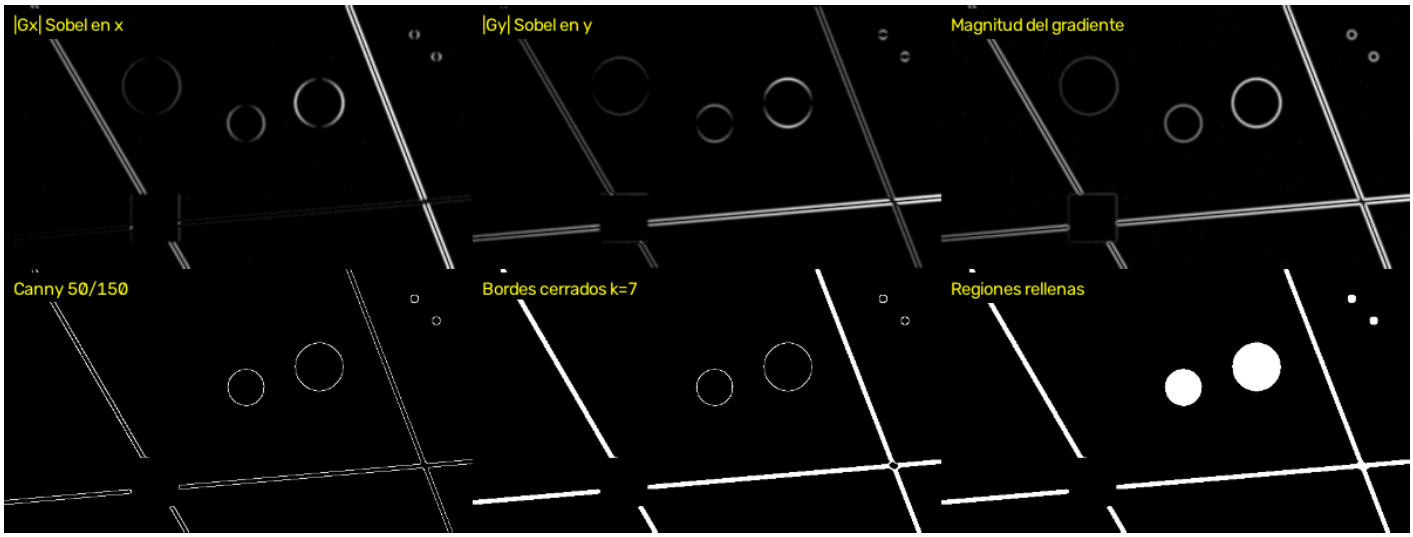

def segmentar_bordes(gris, bajo=50, alto=150, k_cierre=7):
    """Canny -> cerrar huecos -> rellenar contornos externos. Devuelve (bordes, cerrados, regiones)."""
    bordes = cv2.Canny(gris, bajo, alto)                     # 255 en los bordes, 0 en el resto
    elemento = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k_cierre, k_cierre))
    cerrados = cv2.morphologyEx(bordes, cv2.MORPH_CLOSE, elemento)        # une bordes casi tocándose
    contornos, _ = cv2.findContours(cerrados, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)  # solo los de afuera
    regiones = np.zeros_like(gris)
    cv2.drawContours(regiones, contornos, -1, 255, cv2.FILLED)           # -1 = todos; FILLED = rellenos
    return bordes, cerrados, regiones
```

3. Programa principal con barras `Suavizado x10`, `Canny bajo` (50), `Canny alto` (150), `Cierre` (7) y un mosaico de |Gx|, |Gy|, magnitud, Canny, bordes cerrados y regiones rellenas.

> **Sobel con `cv2.CV_32F`:** las derivadas son **negativas** del lado claro-a-oscuro. Si usaras `uint8`, esos valores se perderían (se recortan a 0). Para verlas se usa el valor absoluto y se normalizan a 0..255 (`a_8bits`).

Resultado esperado

```
== Último frame ==
píxeles de borde (Canny): 3283
regiones rellenas       : 6.1 % de la imagen
```



|Gx| marca los bordes verticales y |Gy| los horizontales; Canny da contornos de 1 píxel; al rellenar, los círculos se vuelven regiones, pero la pelota naranja casi no tiene borde en gris

Preguntas del paso 2

1. ¿Por qué la línea horizontal casi no aparece en |Gx| pero sí en |Gy|?
2. Baja el `Suavizado` a 0. ¿Qué le pasa a Canny? ¿Por qué el suavizado de la Práctica 2 es indispensable antes de derivar?
3. Mueve `Canny bajo` y `Canny alto`. ¿Qué cambia con cada uno?
4. ¿Por qué cada línea recta da **dos** bordes paralelos en Canny?
5. La pelota naranja no aparece en “Regiones rellenas”. Mira el gris suavizado y explica por qué.

5. Paso 3 · Agrupación de píxeles (2.7)

Lo que se verá

En vez de un umbral, **agrupamos** píxeles parecidos:

- **k-means de color:** cada píxel es un punto (B, G, R). El algoritmo busca K "centros" de color y asigna cada píxel al más cercano. No le importa **dónde** está el píxel: dos objetos del mismo color quedan en el mismo grupo.
- **Crecimiento de regiones:** desde una **semilla** (un clic), agrega los vecinos cuyo color se parece al de la semilla, y luego los vecinos de esos, etc. Sí importa dónde está: solo crece por píxeles **conectados**.

Qué tienes que hacer

1. Crea `p3_paso3_agrupacion.py` con la `[BASE]` y las funciones:

```python
def segmentar_kmeans(frame, k=4, escala=0.25):
    """k-means sobre los colores (B, G, R) de una versión reducida del frame (para que corra en vivo).
    Devuelve (etiquetas, pintada): el grupo de cada píxel y la imagen pintada con el color de su grupo."""
    chica = cv2.resize(frame, None, fx=escala, fy=escala, interpolation=cv2.INTER_AREA)
    datos = chica.reshape(-1, 3).astype(np.float32)           # una fila por píxel: [B, G, R]
    criterio = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)   # 10 iteraciones o cambio < 1
    _, etiquetas, centros = cv2.kmeans(datos, k, None, criterio, 1, cv2.KMEANS_PP_CENTERS)
    etiquetas = etiquetas.reshape(chica.shape[:2])            # de lista a matriz: grupo de cada píxel
    pintada = centros.astype(np.uint8)[etiquetas]             # cada píxel toma el color (centro) de su grupo
    alto, ancho = frame.shape[:2]
    return (cv2.resize(etiquetas.astype(np.uint8), (ancho, alto), interpolation=cv2.INTER_NEAREST),
            cv2.resize(pintada, (ancho, alto), interpolation=cv2.INTER_NEAREST))


def grupo_de_la_semilla(etiquetas, semilla):
    """Máscara con todos los píxeles del mismo grupo k-means que la semilla (estén donde estén)."""
    x, y = semilla
    return np.where(etiquetas == etiquetas[y, x], 255, 0).astype(np.uint8)


def crecer_region(frame, semilla, tolerancia=30):
    """Crecimiento de regiones con floodFill: desde la semilla, agrega vecinos (4-conectados) cuyo color
    difiere a lo más 'tolerancia' del color de la semilla en cada canal."""
    alto, ancho = frame.shape[:2]
    suave = cv2.GaussianBlur(frame, (5, 5), 0)                # sin suavizar, el ruido corta la región
    mascara = np.zeros((alto + 2, ancho + 2), np.uint8)       # floodFill pide 1 píxel extra por lado
    banderas = 4 | cv2.FLOODFILL_MASK_ONLY | cv2.FLOODFILL_FIXED_RANGE | (255 << 8)
    # 4 = vecindad 4 · MASK_ONLY = no pinta la imagen · FIXED_RANGE = compara con la SEMILLA · 255 = valor
    tol = (tolerancia,) * 3
    cv2.floodFill(suave, mascara, semilla, 0, tol, tol, banderas)
    return mascara[1:-1, 1:-1]                                # quitar el borde extra
```

2. **El clic.** Registra una función que OpenCV llamará con cada evento del mouse:

```python
estado = {"semilla": None, "celda": (320, 180)}

def al_hacer_clic(evento, x, y, banderas, parametro):
    if evento == cv2.EVENT_LBUTTONDOWN:
        ancho_c, alto_c = estado["celda"]                    # tamaño de un panel del mosaico
        estado["semilla"] = ((x % ancho_c) * 2, (y % alto_c) * 2)   # panel a mitad de tamaño

cv2.setMouseCallback(VENTANA, al_hacer_clic)
```

El mosaico muestra cada panel a la **mitad** del tamaño real; por eso se multiplica por 2. Con `%` el clic funciona en cualquiera de los 4 paneles.

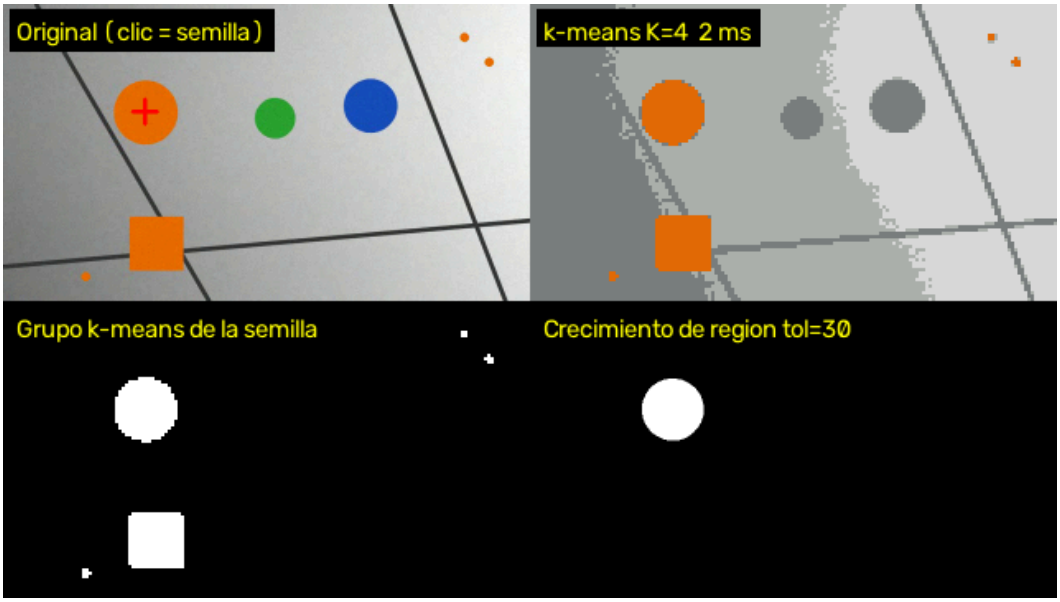
3. En el ciclo: barras `K` (4) y `Tolerancia` (30); k-means en cada frame; si hay semilla, el grupo de la semilla y la región crecida. Mosaico 2×2.

Resultado esperado

Clic sobre la pelota naranja:

```
Haz clic sobre un objeto en el panel 'Original'.
semilla en (172, 130)

== Último frame ==
K = 4   k-means tardó 1.8 ms
grupo de la semilla : 9600 píxeles
región crecida      : 4448 píxeles
```



k-means mete en el mismo grupo la pelota, el cuadrado y los puntos naranjas (mismo color); el crecimiento de regiones solo toma la pelota (está conectada a la semilla)

Preguntas del paso 3

1. Con K = 2, 3, 4 y 6, ¿en qué grupos quedan el fondo, las líneas y cada objeto? ¿Por qué el fondo se parte en varios grupos con K alto?
2. ¿Por qué el grupo k-means de la semilla incluye el cuadrado y los puntos, y la región crecida no?

3. Sube la `Tolerancia` poco a poco hasta 160. ¿A partir de qué valor la región se “derrama” al fondo? ¿Por qué tarda tanto en derramarse? (Pista: compara el canal B de la pelota con el del fondo.) ¿Y con 5?
4. ¿Por qué `crecer_region` suaviza el frame antes de crecer?
5. k-means se calcula sobre el frame reducido a 1/4. ¿Qué ganas y qué pierdes?

6. Paso 4 · Conocimiento del dominio (2.4)

Lo que se verá

Las técnicas anteriores no saben **qué** buscamos. Aquí usamos lo que sabemos del objeto para quedarnos solo con él:

1. **Color:** pasamos a **HSV**, donde el tono (H) no cambia con la iluminación tanto como B, G, R, y nos quedamos con un rango de H, S y V (`cv2.inRange`).
2. **Tamaño:** descartamos contornos con área pequeña (los puntos).
3. **Forma: circularidad** = 4π·área / perímetro^2. Vale 1 en un círculo perfecto y 0.785 en un cuadrado. Descartamos lo que no es redondo (el cuadrado).

Qué tienes que hacer

1. Crea `p3_paso4_dominio.py` con la `[BASE]`, importa `limpiar` del paso 1 y agrega:

```python
from p3_paso1_umbral import limpiar                              # apertura + cierre del paso 1


def mascara_color(frame, h_min, h_max, s_min, v_min):
    """Píxeles cuyo tono está entre h_min y h_max (0..179 en OpenCV) con saturación y brillo suficientes."""
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    return cv2.inRange(hsv, (h_min, s_min, v_min), (h_max, 255, 255))   # 255 dentro del rango, 0 fuera


def describir(contorno):
    """Área, circularidad y rectángulo que encierra al contorno."""
    area = cv2.contourArea(contorno)
    casco = cv2.convexHull(contorno)                             # el borde en píxeles es escalonado y su perímetro
    perimetro = cv2.arcLength(casco, True)                       # sale inflado; el casco convexo lo suaviza
    circularidad = 4 * np.pi * area / perimetro ** 2 if perimetro > 0 else 0
    return area, circularidad, cv2.boundingRect(contorno)     # boundingRect = (x, y, ancho, alto)


def segmentar_dominio(frame, rango=(5, 25, 120, 80), area_min=1500, circ_min=0.90, k=5):
    """Color -> limpieza -> contornos -> filtro por tamaño y forma.
    Devuelve (máscara de color, máscara final, aceptados, rechazados)."""
    color = limpiar(mascara_color(frame, *rango), k)
    contornos, _ = cv2.findContours(color, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    aceptados, rechazados = [], []
    for c in contornos:
        area, circ, caja = describir(c)
        (aceptados if area >= area_min and circ >= circ_min else rechazados).append((c, area, circ, caja))
    final = np.zeros_like(color)
    cv2.drawContours(final, [a[0] for a in aceptados], -1, 255, cv2.FILLED)
    return color, final, aceptados, rechazados
```

2. Barras: `H min` (5), `H max` (25), `S min` (120), `V min` (80), `Area min` (1500) y `Circ x100` (90).

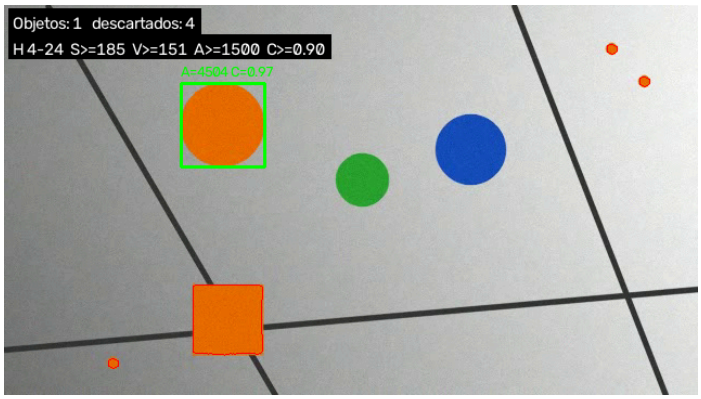
3. **Clic para tomar el color:** en el callback, convierte el píxel clicado a HSV y mueve las barras con `cv2.setTrackbarPos` a ±10 de su tono.
4. Dibuja en **verde** los aceptados (rectángulo con área y circularidad) y en **rojo** los rechazados; escribe el conteo. Segunda ventana con la máscara solo de color y la final.

> **Perímetro del casco convexo.** El contorno de un objeto en píxeles es escalonado, y su perímetro sale inflado (un círculo daría circularidad ≈ 0.8). Medir el perímetro del **casco convexo** (`cv2.convexHull`) suaviza esos escalones: el círculo da ≈ 0.97 y el cuadrado ≈ 0.8.

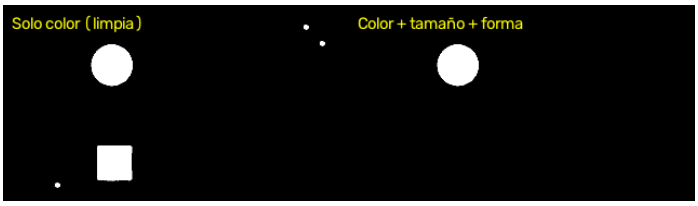
Resultado esperado

```
clic en (142, 154): H=14 S=255 V=231

== Último frame ==
estado        área  circularidad
aceptado      4504          0.97
rechazado       63          0.88
rechazado     4016          0.81
rechazado       64          0.88
rechazado       64          0.88
```



Solo la pelota cumple color + tamaño + forma; el cuadrado y los puntos se descartan (rojo)



Máscara de color (todo lo naranja) y máscara final

Preguntas del paso 4

1. Haz clic sobre el círculo azul y luego sobre el verde. ¿Qué valor de H tiene cada uno? ¿Por qué el rojo es un caso especial en HSV? (Pista: H va de 0 a 179 y es un círculo.)
2. ¿Qué regla elimina los puntos y cuál elimina el cuadrado? Comprueba con los números de la consola.
3. Cuando la pelota pasa sobre el cuadrado, ¿qué ocurre con el conteo? ¿Por qué?
4. Con tu webcam y tu objeto: ¿qué rangos de H, S y V funcionaron? ¿Qué pasa si cambias la iluminación?
5. Propón otra regla del dominio (posición, relación ancho/alto, color promedio…) que ayudaría en tu escena.

7. Paso 5 · Movimiento (2.8)

Lo que se verá

Si la cámara está fija, **lo que cambia es objeto**:

- **Diferencia de frames:** |frame − frame anterior|. Muy simple, pero en un objeto de color uniforme solo cambian sus **orillas**.
- **Fondo promedio:** el fondo es un promedio que "aprende" despacio: `fondo = (1 − α)·fondo + α·frame`. Objeto = lo que difiere del fondo.
- **MOG2:** para cada píxel guarda una **mezcla de gaussianas** (varios valores típicos con su variación). Tolera mejor el ruido y los cambios de luz.

En el video sintético, el **círculo azul se detiene** a la mitad: observa cómo cada método termina "olvidándolo" y lo vuelve fondo.

Qué tienes que hacer

1. Crea `p3_paso5_movimiento.py` con la `[BASE]`, importa `limpiar` y agrega:

```python
from p3_paso1_umbral import limpiar


def diferencia_frames(gris_anterior, gris, umbral=25):
    """Objeto = píxeles que cambiaron más de 'umbral' entre dos frames seguidos."""
    _, mascara = cv2.threshold(cv2.absdiff(gris, gris_anterior), umbral, 255, cv2.THRESH_BINARY)
    return mascara


class FondoPromedio:
    """Fondo = promedio móvil de los frames: fondo = (1 − alfa)·fondo + alfa·frame."""

    def __init__(self):
        self.fondo = None                                   # float32: acumular en uint8 perdería
decimales

    def aplicar(self, gris, alfa=0.02, umbral=25):
        if self.fondo is None:
            self.fondo = gris.astype(np.float32)            # el primer frame es el fondo inicial
        cv2.accumulateWeighted(gris, self.fondo, alfa)      # alfa chica = aprende lento
        fondo_8 = cv2.convertScaleAbs(self.fondo)           # a uint8 para restarlo y verlo
        _, mascara = cv2.threshold(cv2.absdiff(gris, fondo_8), umbral, 255, cv2.THRESH_BINARY)
        return mascara, fondo_8

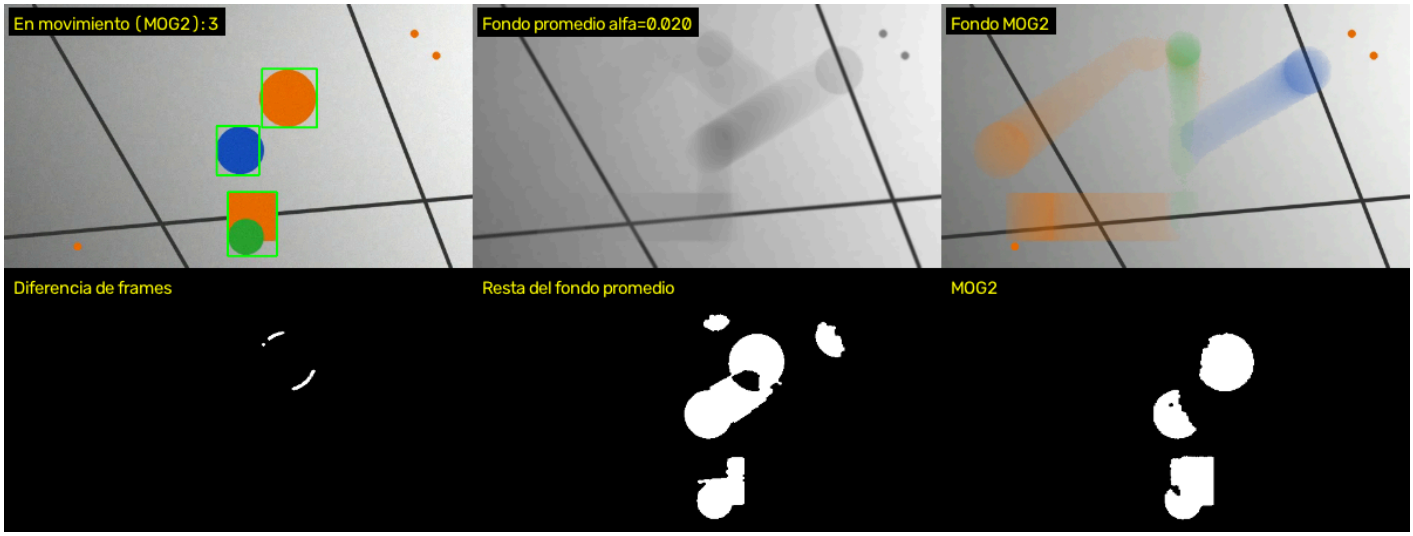

def cajas_en_movimiento(mascara, area_min=300):
    """Rectángulos de las manchas de movimiento con área suficiente."""
    contornos, _ = cv2.findContours(mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    return [cv2.boundingRect(c) for c in contornos if cv2.contourArea(c) >= area_min]
```

2. Antes del ciclo: `promedio = FondoPromedio()` y `mog2 = cv2.createBackgroundSubtractorMOG2(history=300, varThreshold=25, detectShadows=False)`.
3. En el ciclo: gris suavizado, las tres máscaras (limpias), rectángulos verdes sobre las manchas de MOG2 con área ≥ `Area min`, y un mosaico 2×3 con las detecciones, el fondo promedio, el fondo de MOG2

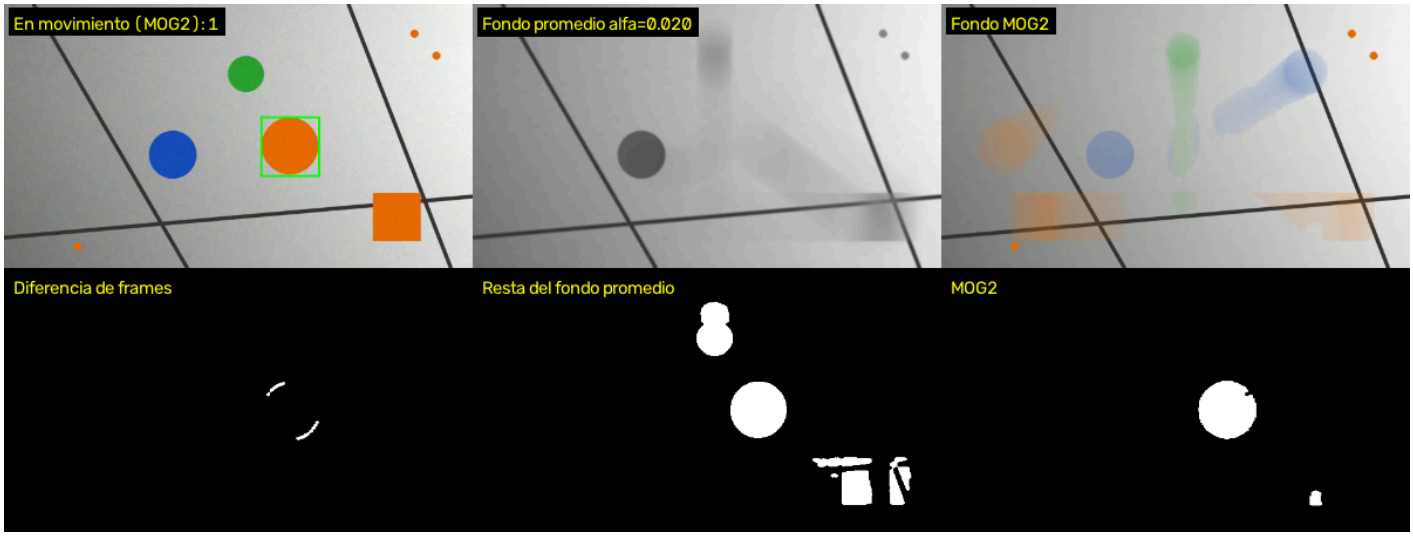
(`mog2.getBackgroundImage()`) y las tres máscaras. Guarda el gris como `anterior` al final de cada vuelta.

Resultado esperado

```
== Último frame ==
diferencia   0.12 % de la imagen en movimiento
promedio     4.91 % de la imagen en movimiento
MOG2         2.24 % de la imagen en movimiento
objetos en movimiento (MOG2): 1
```



Frame 100: todo se mueve. La diferencia de frames solo marca orillas; el fondo promedio deja "estelas" (fantasmas de donde estaban los objetos); MOG2 marca casi todo cada objeto



Frame 230: el círculo azul lleva 80 frames detenido. Los dos modelos ya lo absorbieron como fondo y desaparece de las máscaras

Preguntas del paso 5

1. ¿Por qué la diferencia de frames solo detecta las orillas de los objetos de color uniforme?
2. Cambia `Alfa` a 0.001 y a 0.1. ¿Qué pasa con las "estelas" y con el tiempo que tarda el círculo azul en desaparecer?
3. ¿Cuánto tarda MOG2 en olvidar el círculo azul? ¿Qué parámetro lo controla?

4. Con la webcam: quédate quieto 10 s y luego muévete. Después mueve la cámara. ¿Qué pasa en cada caso?
5. ¿En qué aplicación real es un **problema** que lo quieto se vuelva fondo, y en cuál es una **ventaja**?

8. Paso 6 · Hough y contornos activos (2.9) · *tarea si no se alcanza*

Lo que se verá

Aquí el conocimiento es un **modelo matemático de la forma**:

- **Hough para líneas:** una recta es ρ = x·cos θ + y·sin θ. Cada píxel de borde "vota" por todas las rectas (ρ, θ) que pasan por él; las rectas con más votos existen en la imagen. `HoughLinesP` devuelve segmentos.
- **Hough para círculos:** igual, pero votando por (centro, radio).
- **Contorno activo (*snake*, Kass, Witkin y Terzopoulos, 1988):** una curva cerrada que se deforma para **minimizar su energía**:
  - **interna:** α (elasticidad: que no se estire) y β (rigidez: que no se doble);
  - **externa:** que se coloque sobre bordes fuertes.

  Empieza como un círculo alrededor del objeto y se encoge hasta pegarse a su borde.

Qué tienes que hacer

1. Crea `p3_paso6_hough_snake.py` con la `[BASE]` y agrega:

```python
from skimage.filters import gaussian                              # scikit-image: suavizado para la snake
from skimage.segmentation import active_contour                   # scikit-image: contorno activo (snake)


def lineas_hough(gris, votos=80, largo_min=100):
    """Segmentos de recta: [(x1, y1, x2, y2), ...]."""
    bordes = cv2.Canny(gris, 50, 150)                             # Hough trabaja sobre los bordes
    lineas = cv2.HoughLinesP(bordes, 1, np.pi / 180, votos,   # resolución: 1 px y 1 grado
                             minLineLength=largo_min, maxLineGap=10)
    return [] if lineas is None else [tuple(int(v) for v in l) for l in lineas.reshape(-1, 4)]   # una fila por línea


def circulos_hough(gris, sensibilidad=30, r_min=15, r_max=60):
    """Círculos: [(x, y, r), ...]. Sensibilidad = votos mínimos del acumulador (más bajo = más círculos)."""
    suave = cv2.medianBlur(gris, 5)
    circulos = cv2.HoughCircles(suave, cv2.HOUGH_GRADIENT, dp=1.2, minDist=40,
                                param1=120, param2=sensibilidad, minRadius=r_min, maxRadius=r_max)
    return [] if circulos is None else [tuple(int(v) for v in c) for c in np.round(circulos.reshape(-1, 3))]


def snake(gris, centro, radio, alfa=0.015, beta=10.0, gamma=0.001, puntos=200):
    """Contorno activo que empieza como un círculo. scikit-image usa coordenadas (fila, columna).
    alfa = elasticidad (que no se estire) · beta = rigidez (que no se doble) · gamma = tamaño del paso."""
    t = np.linspace(0, 2 * np.pi, puntos)
    inicial = np.column_stack([centro[1] + radio * np.sin(t), centro[0] + radio * np.cos(t)])   # (y, x)
    suave = gaussian(gris, sigma=3, preserve_range=False)     # la snake necesita bordes suaves para "sentirlos"
    final = active_contour(suave, inicial, alpha=alfa, beta=beta, gamma=gamma)
    return inicial, final


```

```python
def a_puntos_cv(curva):
    """(fila, columna) de scikit-image -> (x, y) enteros de OpenCV para dibujar con polylines."""
    return np.round(curva[:, ::-1]).astype(np.int32).reshape(-1, 1, 2)
```

2. Barras: `Votos` (80), `Largo min` (100), `Sensibilidad` (30), `Radio snake` (60), `Alfa x1000` (15) y `Beta` (10).
3. En cada frame: dibuja las líneas en rojo y los círculos en verde.
4. Con un **clic**, congela el frame y lanza la snake con centro en el clic; muestra en otra ventana el círculo inicial (amarillo) y el contorno final (verde):

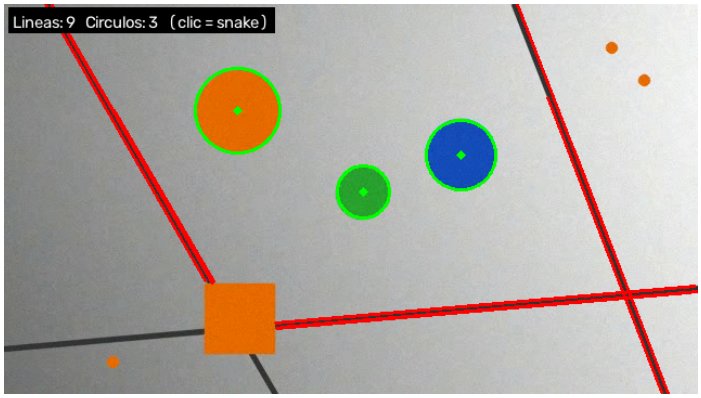
```
inicial, final = snake(gris, centro, radio, alfa, beta)
cv2.polylines(resultado, [a_puntos_cv(inicial)], True, (0, 255, 255), 1)
cv2.polylines(resultado, [a_puntos_cv(final)], True, (0, 255, 0), 2)
```

> **Coordenadas de scikit-image:** usa **(fila, columna)** = (y, x), al revés que OpenCV. `a_puntos_cv` invierte las columnas antes de dibujar.

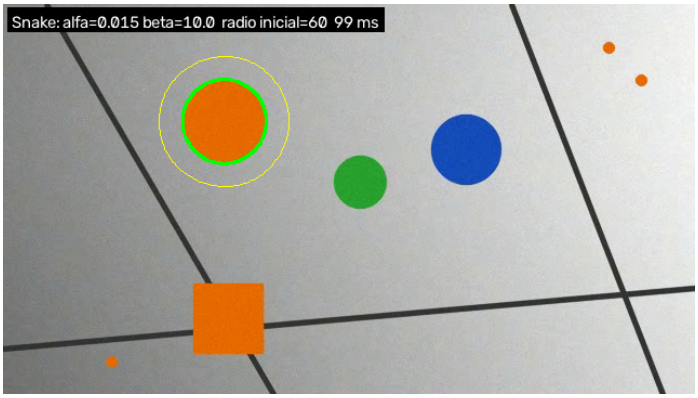
Resultado esperado

```
Haz clic en el centro de un objeto para lanzar la snake.
snake en (204, 108): 99 ms, radio medio final = 38.7 px

== Último frame ==
líneas  : [(250, 294, 637, 260), (255, 298, 639, 265), (44, 1, 192, 257), (38, 0, 185, 255), (473, 0, 558, 220),
(501, 86, 606, 358), (101, 98, 193, 256), (525, 133, 612, 359), (251, 299, 630, 266)]
círculos: [(421, 139, 32), (215, 98, 39), (331, 173, 24)]
```



Hough: 9 segmentos sobre las 3 líneas (cada línea da 2 bordes) y los 3 círculos



Snake: del círculo inicial de radio 60 (amarillo) al borde de la pelota (verde)

Preguntas del paso 6

1. ¿Por qué Hough encuentra más segmentos que líneas hay? ¿Qué parámetro reduce los duplicados?
2. Baja la `Sensibilidad` de círculos a 10. ¿Qué aparece? ¿Detecta el cuadrado como círculo?
3. Lanza la snake con `Alfa` = 0.1 y luego con `Beta` = 0. ¿Cómo cambia la forma final?
4. Lanza la snake con un radio inicial que **no** encierre a la pelota, o con el centro fuera de ella. ¿Qué pasa? ¿Por qué la snake necesita una buena inicialización?
5. Lanza la snake sobre la pelota cuando está en la zona izquierda (fondo oscuro). ¿Funciona? Relaciónalo con la pregunta 5 del paso 2.

9. Paso 7 · Comparativa con IoU · *tarea si no se alcanza*

Lo que se verá

Hasta ahora decidimos "a ojo" qué técnica funciona mejor. Aquí lo **medimos**: el script recorre los 300 frames del video sintético, aplica cada técnica **importándola de su paso** y compara cada máscara con la correcta (`recursos/mascaras.npz`) usando **IoU** y **Dice**.

Se evalúan dos tareas:

- **A. Todos los objetos contra el fondo.**
- **B. Solo la pelota naranja.** Las técnicas que necesitan un clic usan como semilla el centro verdadero de la pelota.

Qué tienes que hacer

1. Crea `p3_paso7_comparativa.py` con la `[BASE]` e importa las funciones de los pasos anteriores:

```python
import matplotlib.pyplot as plt                                  # gráficas de resultados

from p3_paso1_umbral import (limpiar, preparar_gris, segmentar_adaptativo, segmentar_global,
                             segmentar_otsu)
from p3_paso2_bordes import segmentar_bordes
from p3_paso3_agrupacion import crecer_region, grupo_de_la_semilla, segmentar_kmeans
from p3_paso4_dominio import mascara_color, segmentar_dominio
from p3_paso5_movimiento import FondoPromedio
from p3_paso6_hough_snake import circulos_hough

RUTA_MASCARAS = CARPETA / "recursos" / "mascaras.npz"
CALENTAMIENTO = 30                                               # frames para que los modelos de fondo aprendan


def iou(a, b):
    """Intersección sobre unión de dos máscaras 0/255."""
    a, b = a > 0, b > 0
    union = np.logical_or(a, b).sum()
    return np.logical_and(a, b).sum() / union if union else 1.0


def dice(a, b):
    a, b = a > 0, b > 0
    total = a.sum() + b.sum()
    return 2 * np.logical_and(a, b).sum() / total if total else 1.0


def centro_de(mascara):
    """Centro (x, y) de una máscara, con momentos de imagen. Hace las veces del clic del usuario."""
    m = cv2.moments(mascara, binaryImage=True)
    return int(m["m10"] / m["m00"]), int(m["m01"] / m["m00"])


def circulo_mas_cercano(gris, semilla):
    """Máscara del círculo de Hough más cercano a la semilla (vacía si no hay círculos)."""
    mascara = np.zeros_like(gris)
    circulos = circulos_hough(gris)
    if circulos:
```

```
        x, y, r = min(circulos, key=lambda c: (c[0] - semilla[0]) ** 2 + (c[1] - semilla[1]) ** 2)
        cv2.circle(mascara, (x, y), r, 255, -1)
    return mascara
```

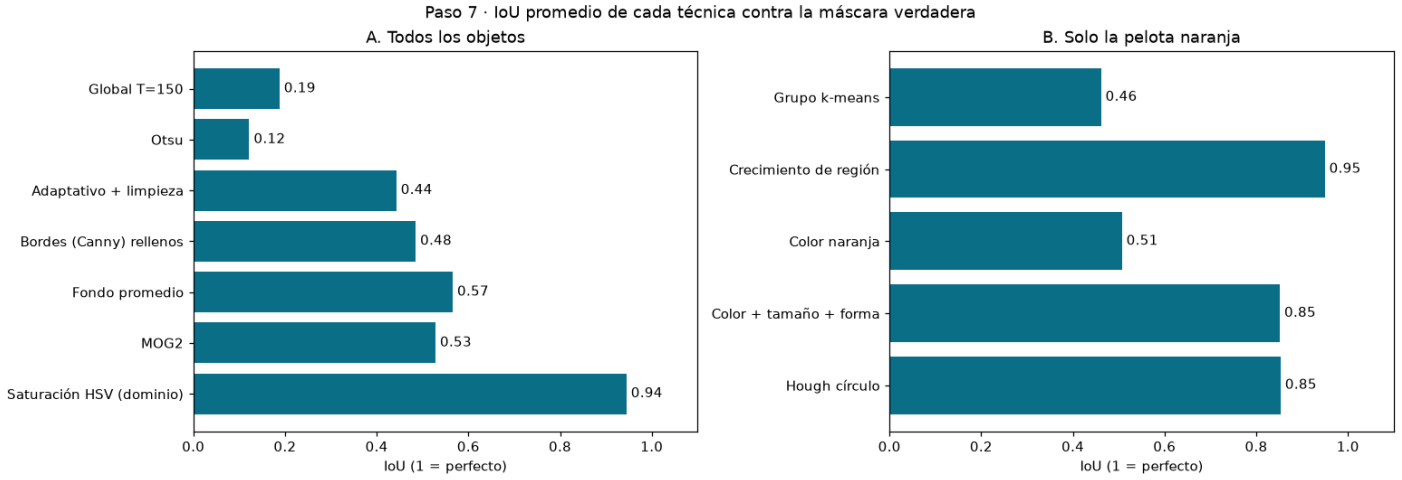
2. En el programa principal:
   a. carga las máscaras con `verdad = dict(np.load(RUTA_MASCARAS))` (el `dict` las descomprime una sola vez; sin él, cada acceso descomprime todo el archivo y el script tarda minutos);
   b. recorre el video frame por frame y aplica cada técnica con parámetros fijos;
   c. a partir del frame 30 (para que los modelos de fondo aprendan), guarda el IoU y el Dice de cada técnica contra `verdad["objetos"][n]` o `verdad["pelota"][n]`;
   d. imprime la tabla y haz dos figuras: barras con el IoU promedio y las máscaras del frame 200.

Resultado esperado

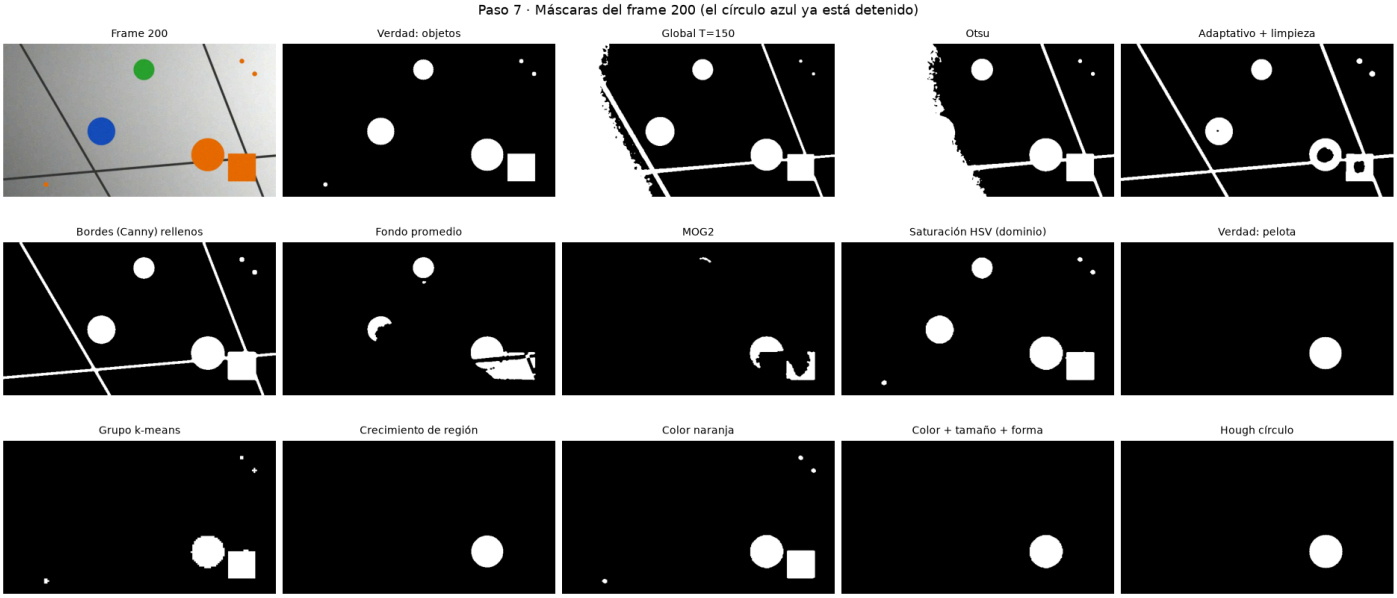
```
== Resultados en 270 frames (del 30 al 299) ==

A. Todos los objetos
método                        IoU   Dice  ms/frame
Global T=150                0.188  0.316      0.03
Otsu                        0.122  0.217      0.09
Adaptativo + limpieza       0.443  0.612      3.05
Bordes (Canny) rellenos     0.485  0.642      0.94
Fondo promedio              0.566  0.718      0.32
MOG2                        0.528  0.673      1.31
Saturación HSV (dominio)    0.945  0.972      0.45

B. Solo la pelota naranja
método                        IoU   Dice  ms/frame
Grupo k-means               0.461  0.617      2.34
Crecimiento de región       0.949  0.971      0.26
Color naranja               0.507  0.671      0.56
Color + tamaño + forma      0.851  0.873      0.65
Hough círculo               0.853  0.874      3.62
```



IoU promedio de cada técnica



Máscaras de cada técnica en el frame 200

Preguntas del paso 7

1. En la tarea A, ¿por qué la saturación HSV le gana por mucho a todas las técnicas basadas en gris? ¿Funcionaría igual si los objetos fueran grises?
2. ¿Por qué el fondo promedio y MOG2 no llegan a 0.6 de IoU? (Pista: el círculo azul y los puntos.)
3. En la tarea B, ¿por qué “Color naranja” y “Grupo k-means” se quedan en ~0.5 y “Color + tamaño + forma” sube a ~0.85? ¿En qué frames falla este último?
4. El crecimiento de regiones es el mejor en B. ¿Qué ventaja injusta tiene en esta comparación?
5. ¿Qué técnica elegirías para contar pelotas naranjas en una banda transportadora con iluminación controlada? Justifica con IoU **y** con ms/frame.

10. Retos y entregable

Retos

Reto	Qué hacer	Puntos
Snake programada a mano	Programa tu propia snake **sin scikit-image**: una lista de puntos que en cada iteración se mueven al vecino (de 8) con menor energía = continuidad + curvatura − gradiente (algoritmo voraz de Williams y Shah, 1992). Muestra el antes y el después y compárala con la de scikit-image	**+2**
Snake morfológica	Usa `morphological_geodesic_active_contour` de scikit-image (con `disk_level_set` como inicio y `balloon=-1`). Compárala con la snake clásica: ¿cuál es más robusta y cuál más rápida?	+0.5
Región de interés	En el paso 4, agrega una regla de **posición**: solo cuenta objetos dentro de un rectángulo dibujado con dos clics	+0.5
Contador de cruces	Con el paso 5, cuenta cuántos objetos cruzan una línea vertical a la mitad de la imagen	+0.5

> **Pistas para la snake a mano.** Representa la snake como un arreglo `(N, 2)` de enteros (x, y). Calcula la magnitud del gradiente de la imagen **una vez**. Para cada punto prueba sus 9 posiciones (incluida la actual) y normaliza cada término de energía a 0..1 antes de sumarlos. Sin una fuerza que la "desinfle" hacia adentro (fuerza globo), la snake casi no se mueve si empieza lejos del borde.

Entregable

Sube **un** `.zip` con los 7 scripts (y los de retos) y un **reporte PDF** (máximo 6 páginas) con:

- capturas de los pasos 1–6 con **tu webcam y tu objeto**, además de las del video sintético;
- la tabla de IoU del paso 7 y tu análisis;
- las respuestas a las preguntas.

Criterio	Puntos
Los scripts corren con `--video` y con webcam, importando las funciones de los pasos anteriores	4
Código ordenado y comentado; misma `[BASE]`	1
Respuestas correctas y justificadas con las figuras y las métricas	3
Análisis de la tabla de IoU y elección justificada de técnica	1
Reporte claro con capturas de tu cámara	1
Retos (extra)	hasta +3

11. Errores frecuentes

Síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'p3_paso1_umbral'`	Los scripts no están en la misma carpeta, o se ejecuta desde otra	Todos en la misma carpeta; ejecuta desde ella
`ModuleNotFoundError: No module named 'skimage'`	Falta scikit-image en el entorno	`pip install scikit-image` con el entorno activo
Al importar un paso se abre su ventana	Falta el `if __name__ == "__main__":` en el paso importado	Todo el programa principal va dentro de ese `if`
`adaptiveThreshold` falla	`blockSize` par o menor que 3	`bloque = 2 * valor + 1` con valor ≥ 1
La máscara de color sale vacía	El tono H de OpenCV va de 0 a **179**, no de 0 a 360	Divide entre 2 los valores de otras herramientas; usa el clic
La snake se va a un lugar raro	Coordenadas (x, y) en lugar de (fila, columna), o imagen sin suavizar	`inicial = [y, x]`; usar `gaussian(gris, 3)`
El paso 7 tarda minutos	Se lee `np.load(...)["pelota"]` en cada frame	`verdad = dict(np.load(...))` una sola vez

Referencia rápida

Función	Qué hace
`cv2.threshold(g, T, 255, cv2.THRESH_BINARY_INV [+ cv2.THRESH_OTSU])`	Umbral global u Otsu → `(T, máscara)`
`cv2.adaptiveThreshold(g, 255, ADAPTIVE_THRESH_GAUSSIAN_C, THRESH_BINARY_INV, bloque, C)`	Umbral adaptativo
`cv2.morphologyEx(m, cv2.MORPH_OPEN / MORPH_CLOSE, elemento)`	Apertura / cierre
`cv2.Sobel(g, cv2.CV_32F, dx, dy)`, `cv2.magnitude(gx, gy)`, `cv2.Canny(g, bajo, alto)`	Gradiente y bordes
`cv2.findContours(m, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)`, `cv2.drawContours(..., FILLED)`	Contornos y relleno
`cv2.contourArea`, `cv2.arcLength`, `cv2.convexHull`, `cv2.boundingRect`	Medir contornos
`cv2.kmeans(datos, K, None, criterio, intentos, KMEANS_PP_CENTERS)`	k-means
`cv2.floodFill(img, mascara, semilla, 0, tol, tol, banderas)`	Crecimiento de región
`cv2.cvtColor(f, COLOR_BGR2HSV)`, `cv2.inRange(hsv, bajo, alto)`	Segmentación por color
`cv2.setMouseCallback(ventana, funcion)`, `cv2.setTrackbarPos`	Clics y mover barras desde el código
`cv2.accumulateWeighted`, `cv2.createBackgroundSubtractorMOG2`	Modelos de fondo
`cv2.HoughLinesP`, `cv2.HoughCircles`	Hough para líneas y círculos
`skimage.segmentation.active_contour(img, inicial, alpha, beta, gamma)`	Snake clásica